

ML System Architecture

Best practices for designing machine learning systems

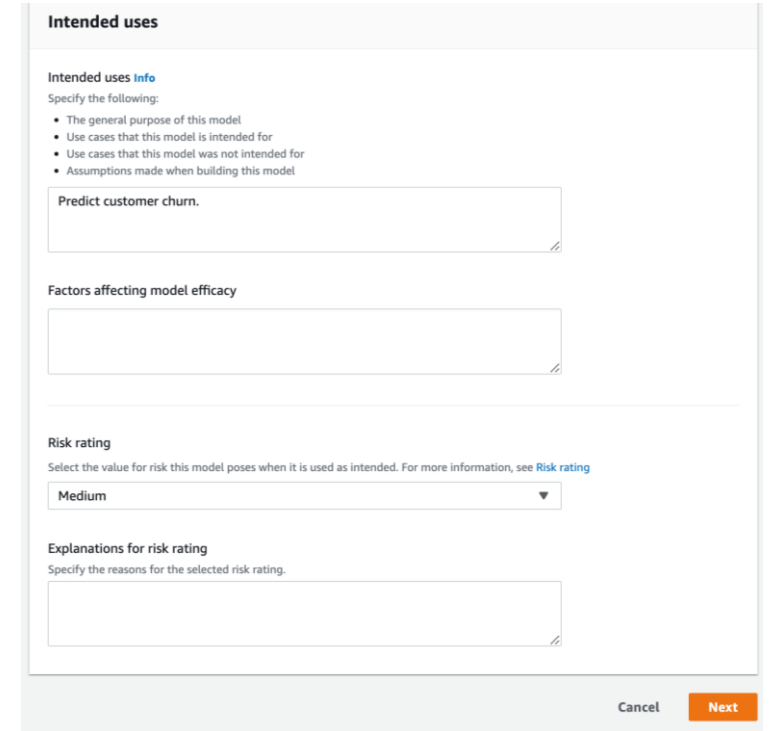
Responsible AI: Core Dimensions

- Fairness
- Explainability
- Privacy and Security
- Safety
- Controllability
- Veracity and Robustness
- Governance
- Transparency



AWS Tools for Responsible AI

- Amazon Bedrock
 - Model evaluation tools
- SageMaker Clarify
 - Bias detection
 - Model evaluation
 - Explainability
- SageMaker Model Monitor
 - Get alerts for inaccurate responses
- Amazon Augmented AI
 - Insert humans in the loop to help correct results
- SageMaker ML Governance
 - SageMaker Role Manager
 - Model Cards
 - Model Dashboard



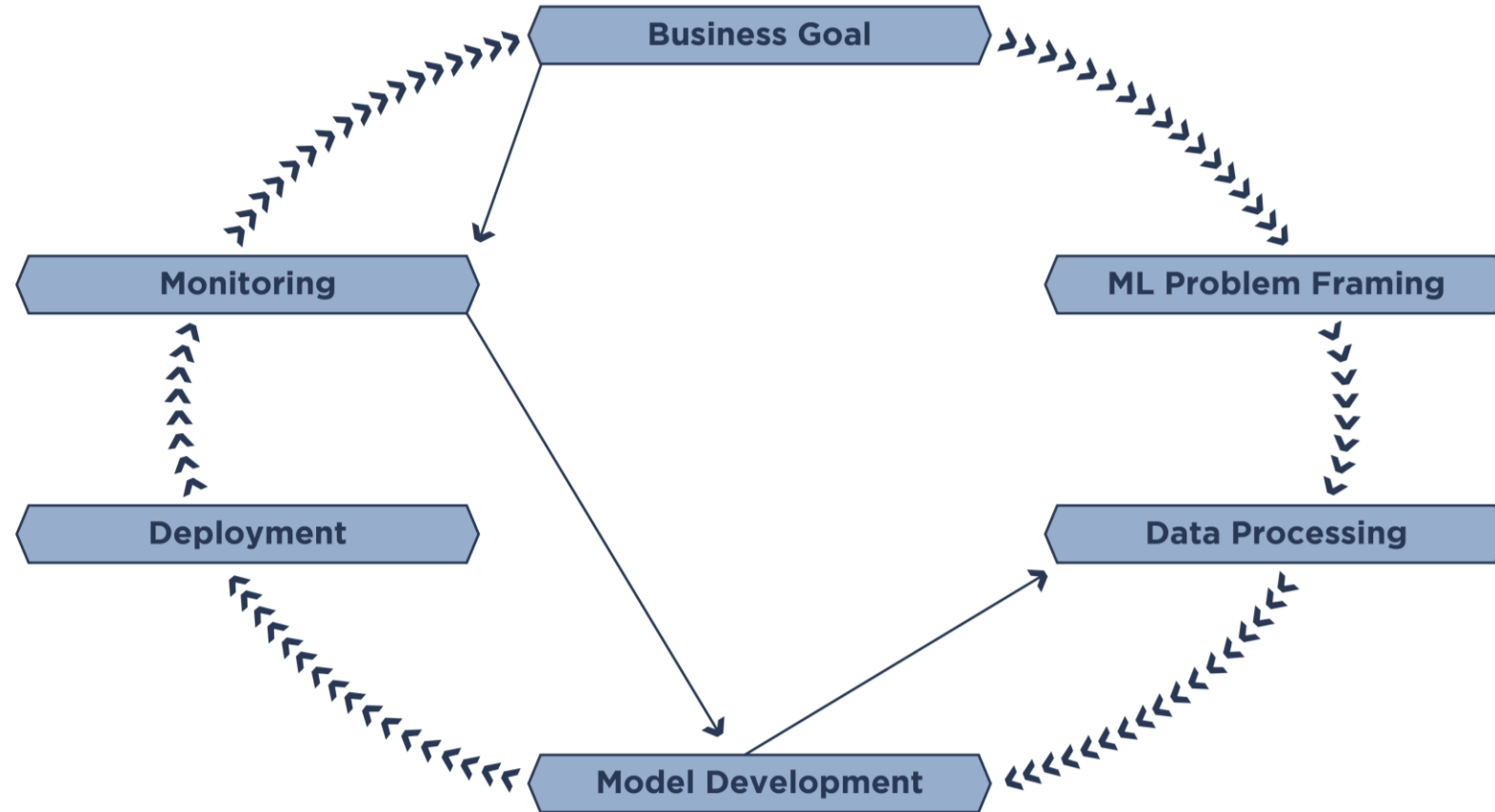
The screenshot shows a form titled "Intended uses" with a sub-header "Intended uses [Info](#)". Below this, it says "Specify the following:" and lists three bullet points: "The general purpose of this model", "Use cases that this model is intended for", and "Use cases that this model was not intended for". There is a text input field with the placeholder text "Predict customer churn." and a small "Info" icon. Below this is another text input field with the placeholder text "Factors affecting model efficacy" and a small "Info" icon. Further down is a "Risk rating" section with a dropdown menu set to "Medium" and a link to "Risk rating". Below that is an "Explanations for risk rating" section with a text input field and a small "Info" icon. At the bottom right are "Cancel" and "Next" buttons.

Portion of model card creation UI

ML Design Principles

- Assign ownership
- Provide protection
 - Security controls
- Enable resiliency
 - Fault tolerance
 - Recoverability
- Enable reusability
- Enable reproducibility
 - Version control
- Optimize resources
- Reduce cost
- Enable automation
 - CI/CD, CT (training)
- Enable continuous improvement
 - Monitoring & analysis
- Minimize environmental impact
 - Sustainability
 - Managed services
 - Efficient hardware and software

The Machine Learning Lifecycle (As defined by AWS)

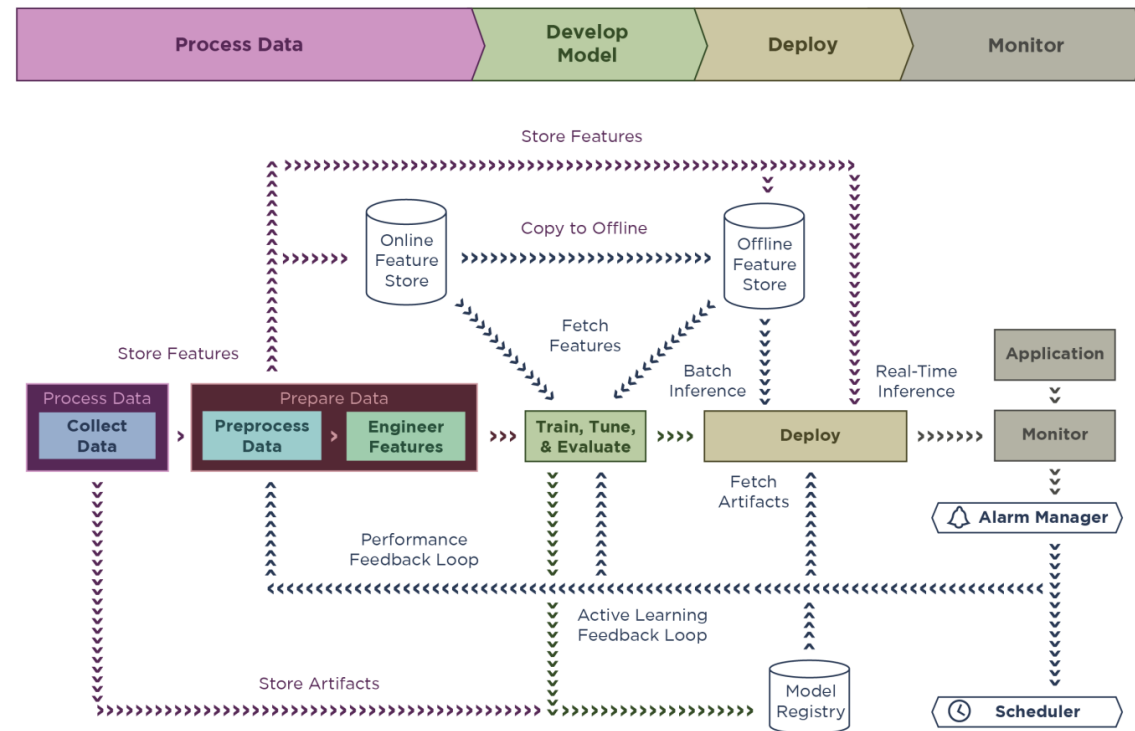


Business Goal Identification

- Develop the right skills with accountability and empowerment
- Discuss and agree on the level of model explainability
 - SageMaker Clarify can help
- Monitor model compliance to business requirements
 - Figure out what to monitor and how to measure drift
- Validate ML data permissions, privacy, software, and license terms
- Determine key performance indicators
- Define overall return on investment (ROI) and opportunity cost
- Use managed services to reduce total cost of ownership (TCO)
- Define the overall environmental impact or benefit



ML Problem Framing: The ML Lifecycle



Frame ML Problem

- Establish ML roles and responsibilities
 - SageMaker Role Manager
- Prepare an ML profile template
 - Document the resources required
- Establish model improvement strategies
 - SageMaker Experiments, hyper-parameter optimization, AutoML
- Establish a lineage tracker system
 - SageMaker Lineage Tracking, Pipelines, Studio, Feature Store, Model Registry
- Establish feedback loops across ML lifecycle phases
 - SageMaker Model Monitor, CloudWatch, Amazon Augmented AI (A2I)
- Review fairness and explainability (SageMaker Clarify)



Frame ML Problem

- Design data encryption and obfuscation (Glue DataBrew)
- Use APIs to abstract change from model consuming applications
 - SageMaker + API Gateway
- Adopt a machine learning microservice strategy
 - Lambda, FarGate
- Use purpose-built AI and ML services and resources
 - SageMaker, JumpStart, marketplace
- Define relevant evaluation metrics
- Identify if machine learning is the right solution
- Tradeoff analysis on custom versus pre-trained models



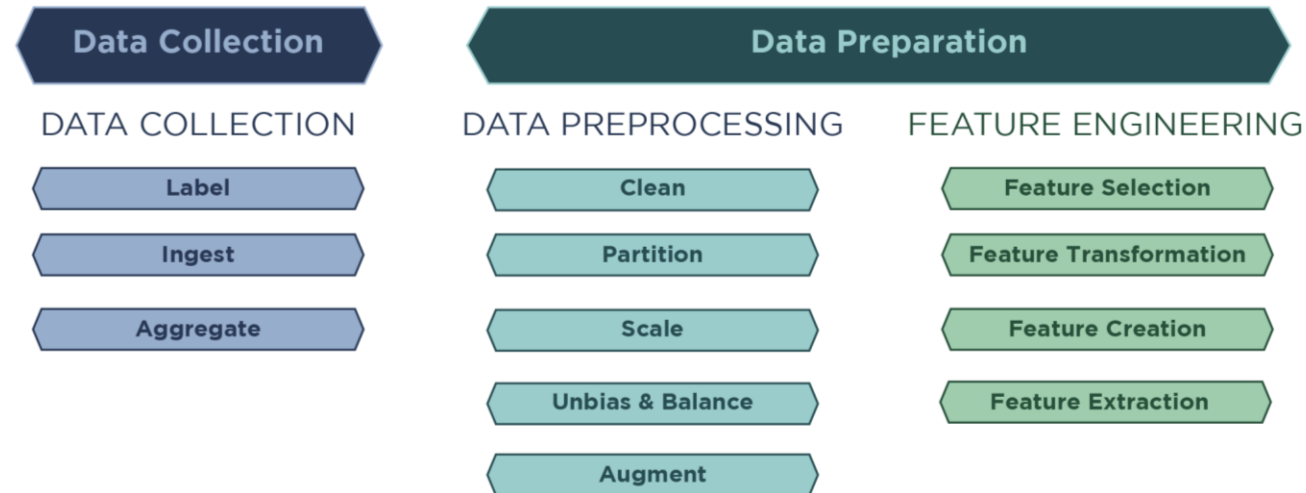
Frame ML Problem

- Consider AI services and pre-trained models
 - Training large AI models is very resource and energy-intensive
 - Can you re-use an existing system that's out there?
- Select sustainable regions
 - Where you perform your processing, training, and deployment may affect sustainability
 - Some regions have “greener” energy than others



Data Processing

DATA PROCESSING



Data Processing

- Profile data to improve quality
 - Amazon's data engineering & analysis tools
 - Data Wrangler, Glue, Athena, Redshift, Quicksight...
- Create tracking and version control mechanisms
 - SageMaker model registry, store notebooks in git, SageMaker Experiments
- Ensure least privilege access
- Secure data and modeling environment
 - Use analysis environments in the cloud (SageMaker, EMR, Athena...)
 - IAM, KMS, Secrets Manager, VPC's / PrivateLink
- Protect sensitive data privacy (Macie)
- Enforce data lineage (SageMaker ML Lineage Tracker)
- Keep only relevant data
 - Remove PII with Comprehend, Transcribe, Athena...



Data Processing

- Use a data catalog (AWS Glue)
- Use a data pipeline (SageMaker Pipelines)
- Automate managing data changes (MLOps)
- Use a modern data architecture (data lake)
- Use managed data labeling (Ground Truth)
- Use data wrangler tools for interactive analysis (Data Wrangler)
- Use managed data processing capabilities (SageMaker)
- Enable feature reusability (feature store)

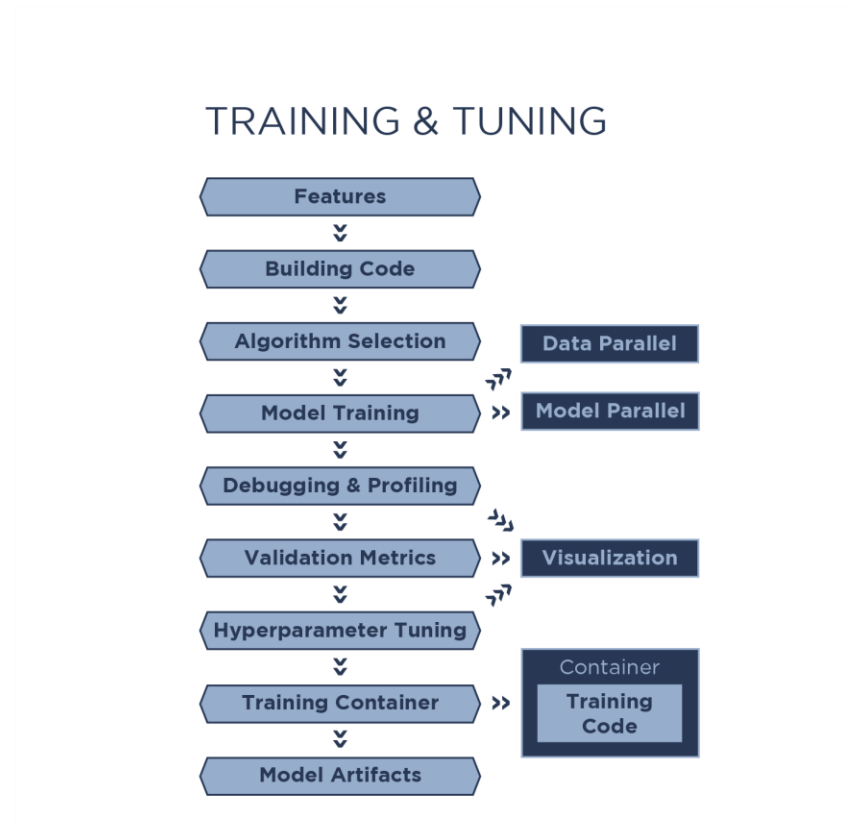


Data Processing

- Minimize idle resources
- Implement data lifecycle policies aligned with sustainability goals
- Adopt sustainable storage options



Model Development: Training and Tuning



Model Development

- Automate operations through MLOps and CI/CD
 - CloudFormation, CDK, SageMaker Pipelines, Step Functions
- Establish reliable packaging patterns to access approved public libraries
 - ECR, CodeArtifact
- Secure governed ML environment
- Secure inter-node cluster communications
 - SageMaker inter-node encryption, EMR encryption in transit
- Protect against data poisoning threats
 - SageMaker Clarify, rollback with SageMaker Model Registry & Feature Store
- Enable CI/CD/CT automation with traceability
 - MLOps Framework, SageMaker Pipelines



Model Development

- Ensure feature consistency across training and inference
 - SageMaker Feature Store
- Ensure model validation with relevant data
 - SageMaker Experiments, SageMaker Model Monitor
- Establish data bias detection and mitigation
 - SageMaker Clarify
- Optimize training and inference instance types
- Explore alternatives for performance improvement
 - SageMaker Experiments



Model Development

- Establish a model performance evaluation pipeline
 - SageMaker Pipelines, Model Registry
- Establish feature statistics
 - Data Wrangler, Model Monitor, Clarify, Experiments
- Perform a performance trade-off analysis
 - Accuracy vs. complexity
 - Bias vs. fairness
 - Bias vs. variance
 - Precision vs. recall
 - Test with Experiments and Clarify



Model Development

- Detect performance issues when using transfer learning
 - SageMaker Debugger
- Select optimal computing instance size
- Use managed build environments
- Select local training for small scale experiments
- Select an optimal ML framework (PyTorch, Tensorflow, scikit-learn)
- Use automated machine learning (SageMaker Autopilot)



Model Development

- Use managed training capabilities
 - SageMaker, Training Compiler, managed Spot Instances
- Use distributed training
 - SageMaker Distributed Training Libraries
- Stop resources when not in use
 - Billing alarms, SageMaker Lifecycle Configuration, SageMaker Studio auto-shutdown
- Start training with small datasets
- Use warm-start and checkpointing hyperparameter tuning
- Use hyperparameter optimization technologies
 - SageMaker automatic model tuning



Model Development

- Setup budget and use resource tagging to track costs
 - AWS Budgets, Cost Explorer
- Enable data and compute proximity
- Select optimal algorithms
- Enable debugging and logging
 - SageMaker Debugger, CloudWatch



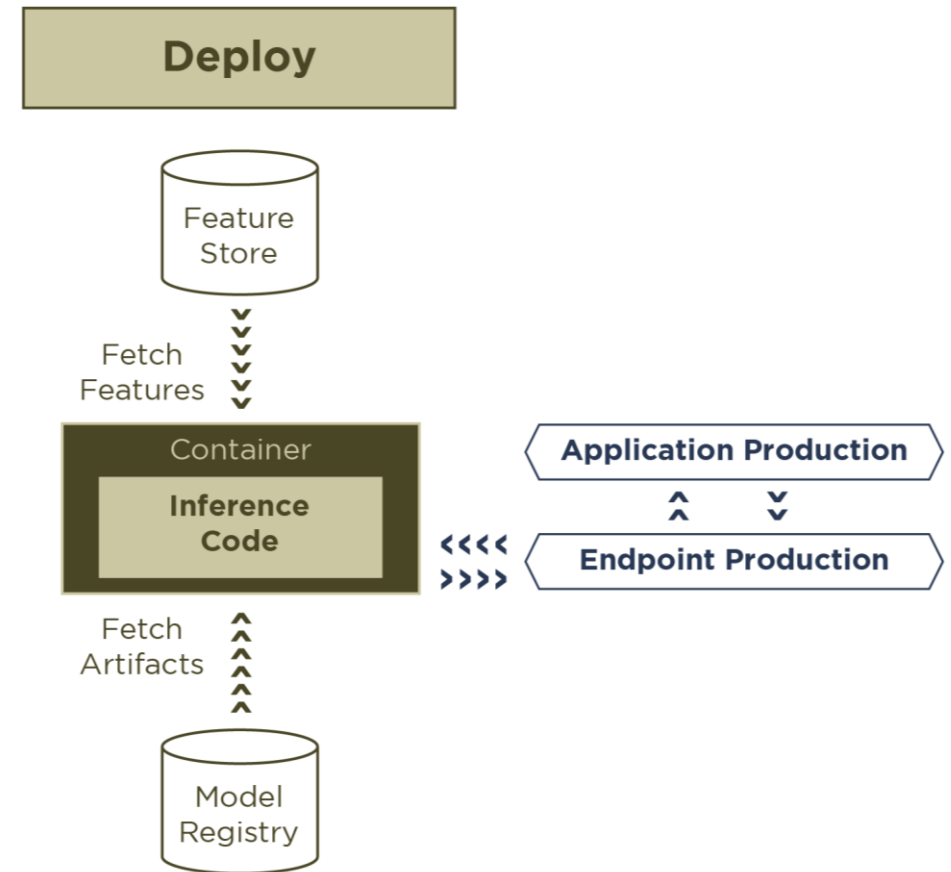
Model Development

- Define sustainable performance criteria
 - How much accuracy is enough?
 - Early stopping
- Select energy-efficient algorithms
 - Do you really need a large model?
- Archive or delete unnecessary training artifacts
 - SageMaker Experiments can help organize them
- Use efficient model tuning methods
 - Bayesian or hyperband, not random or grid search
 - Limit concurrent training jobs
 - Tune only the most important hyperparameters



Deployment

- Establish deployment environment metrics
 - CloudWatch, EventBridge, SNS
- Protect against adversarial and malicious activities
 - SageMaker Model Monitor
- Automate endpoint changes through a pipeline
 - SageMaker Pipelines
- Use an appropriate deployment and testing strategy
 - SageMaker Blue/Green deployments, A/B Testing, linear deployments, canary deployments



Deployment

- Evaluate cloud vs. edge options
 - SageMaker Neo, IoT Greengrass
- Choose an optimal deployment option in the cloud
 - Real-time, serverless, asynchronous, batch
- Use appropriate deployment option
 - Multi-model, multi-container, SageMaker Edge
- Explore cost effective hardware options
 - Elastic Inference, Inf1 instances, SageMaker Neo
- Right-size the model hosting instance fleet
 - SageMaker Inference Recommender, AutoScaling



Deployment

- Align SLAs with sustainability goals
 - Latency vs. serverless / batch / asynchronous deployments
- Use efficient silicon
 - Graviton3 for CPU inference
 - Inferentia (inf2) for deep learning inference
 - Trainium (trn1) for training
- Optimize models for inference
 - Neo, Treelite, Hugging Face Infinity
- Deploy multiple models behind a single endpoint
 - SageMaker Inference Pipelines



Monitoring

- Enable model observability and tracking
 - SageMaker Model Monitor, CloudWatch, Clarify, Model Cards, lineage tracking
- Synchronize architecture and configuration, and check for skew across environments
 - CloudFormation, Model Monitor
- Restrict access to intended legitimate consumers
 - Secure inference endpoints
- Monitor human interactions with data for anomalous activity
 - Logging, GuardDuty, Macie



Monitoring

- Allow automatic scaling of the model endpoint
 - Auto-scaling, Elastic Inference
- Ensure a recoverable endpoint with a managed version control strategy
 - SageMaker Pipelines & Projects, CodeCommit, CloudFormation, ECR
- Evaluate model explainability
- Evaluate data drift
- Monitor, detect, and handle model performance degradation
 - Clarify, Model Monitor, OpenSearch



Monitoring

- Establish an automated re-training framework
 - SageMaker Pipelines, Step Functions, Jenkins
- Review updated data/features for retraining
 - SageMaker Data Wrangler
- Include human-in-the-loop monitoring
 - Amazon Augmented AI



Monitoring

- Monitor usage and cost by ML activity
 - Tagging, Budgets
- Monitor return on investment for ML models
 - Quicksight
- Monitor endpoint usage and right-size the instance fleet
 - CloudWatch
 - SageMaker autoscaling
 - Use Amazon FSx for Lustre instead of S3 for training?



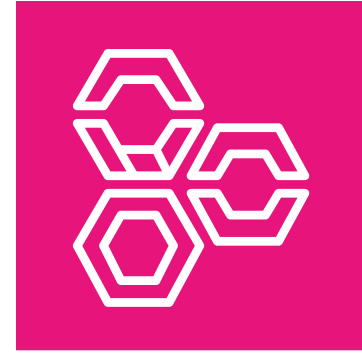
Monitoring

- Measure material efficiency
 - Measure provisioned resources / business outcomes
- Retrain only when necessary
 - Define what accuracy is acceptable, only retrain when in violation
 - SageMaker Model Monitor
 - SageMaker Pipelines
 - AWS Step Functions Data Science SDK



AWS Well-Architected Machine Learning (ML) Lens

- The principles of this section are embodied in a “custom lens” for the AWS Well-Architected Tool
- This is, in practice, a JSON file that leads you through an interview about how you are using ML
 - And guides you on how you might do it better.
- Associated white paper:
 - <https://docs.aws.amazon.com/wellarchitected/latest/machine-learning-lens/machine-learning-lens.html>



AWS Well-Architected Tool

MLOps Workload Orchestrator on AWS

- From AWS Solutions Library
- CloudFormation template
- Can set up training, inference, monitoring
- Currently 12 pipelines to pick from
 - Training with SageMaker and hyperparameter tuning or autopilot
 - “Bring-your-own model” (BYOM) real-time or batch inference with SageMaker
 - Custom algorithm pipeline for your own Docker / ECR images
 - Model monitor pipelines on inference for data quality, model quality, bias, and explainability



MLOps Workload Orchestrator

